# LABORATORY EXPERIMENTAL PROTOCOL

Applied Massive Parallelism: CUDA Python, Thread Geometries, and Hardware Latency Pitfalls

| | | | |
|---|---|---|---|
| **Student Name:** | _Bagdel Aizhan___ | **Student ID:** | _230107088 |
| **Session / Date:** | **10/09/2026 ,01-N** | **Assigned GPU ID:** | _____ ___ |

## SECTION 1: ENVIRONMENT PROVISIONING & HARDWARE SETUP (15

**MANDATORY SUBMISSION FIELD: Fill in the exact telemetry values printed by your instance below:**

| | |
|---|---|
| **NVIDIA Driver Version (from nvidia-smi table)** | [ Record: 580.82.07 ] |
| **CUDA Version Supported by Driver** | [ Record: __13.0 _ ] |
| **Allocated GPU Name (e.g., Tesla T4)** | [ Record: ___Tesla t4_ ] |
| **Total Available VRAM (e.g., 15360 MiB)** | [ Record: __15360 MiB _ ] |
| **Compute Capability Major.Minor (e.g., 7.5)** | [ Record: _**7.5** (printed as `major=7, minor=5`)__ ] |
| **Max Threads Per Block (hardware limit)** | [ Record: ___1024 _ ] |

## SECTION 2: THE LIE OF GPU SPEED & THE CROSSOVER EXPERIMENT (25 MIN)

Novice programmers assume: 'If I put my code on the GPU, it will instantly run 100x faster.' This is false. A GPU is connected to the CPU via the PCIe bus—a narrow physical copper bus with high latency. If the data is small, the overhead of copying bytes to the GPU dwarfs the computation time. In this experiment, you will prove this empirically.

### Task 2.1: Implement the Vector Doubling Kernels

Add a new code cell and paste the following baseline comparison framework:

```
import numpy as np
import time
from numba import cuda

# 1. CPU Reference Implementation (Pure Python loop)
def double_cpu_pure(arr):
 out = np.empty_like(arr)
 for i in range(len(arr)):
 out[i] = arr[i] * 2.0
 return out

# 2. GPU Kernel Implementation
@cuda.jit
def double_gpu_kernel(d_in, d_out):
 # Calculate unique 1D thread ID
 idx = cuda.grid(1)
 if idx < d_in.shape[0]:
 d_out[idx] = d_in[idx] * 2.0
```

### Task 2.2: Benchmark Across Orders of Magnitude

Execute the benchmark harness across different array sizes N from 100 up to 10,000,000. For each N, measure two distinct GPU metrics:

**1. Total Roundtrip GPU Time:** Host-to-Device copy (PCIe) + Kernel Execution + Device-to-Host copy (PCIe).



**2. Kernel-Only Compute Time:** Data is already pre-loaded into VRAM; measure ONLY the GPU

**MANDATORY EXPERIMENTAL DATA TABLE: Record your empirical times (in milliseconds) below:**

```
•••         100 | CPU      0.101 | GPU total      2.049 | kernel      0.143
        1,000 | CPU      0.589 | GPU total      0.788 | kernel      0.113
       10,000 | CPU      5.677 | GPU total      0.796 | kernel      0.119
      100,000 | CPU     55.482 | GPU total      0.893 | kernel      0.105
    1,000,000 | CPU    521.598 | GPU total      5.529 | kernel      0.146
    5,000,000 | CPU        nan | GPU total     18.294 | kernel      0.297
   10,000,000 | CPU        nan | GPU total     25.985 | kernel      0.462
```



The CPU was not measured at these sizes. The GPU is the winner by extrapolation, since the CPU took 521 ms at 1M and grows linearly.

## Q1: crossover point

At N = 1,000 the CPU still wins (0.589 < 0.788). At N = 10,000 the GPU wins for the first time (0.796 < 5.677) and stays ahead at every larger size.

**N* = 10,000.** Use `CROSSOVER_N = 10000` in Section 6.

> The empirical crossover point on my Tesla T4 was N* = 10,000. At N = 100 and 1,000 the CPU was faster (0.101 ms and 0.589 ms versus 2.049 ms and 0.788 ms for GPU Total), because the fixed PCIe transfer and launch overhead exceeded the computation. At N = 10,000 the GPU Total (0.796 ms) beat the CPU (5.677 ms), and the gap kept growing, reaching about 94x at N = 1,000,000 (521.598 ms vs 5.529 ms).

### Q2: why the GPU is slower at N = 100

(2.049 − 0.143) / 2.049 ≈ **93.0%** of the time was overhead. This is slightly under the sheet's "95%+", so report your measured value.

> At N = 100, GPU Total (2.049 ms) is about 20x slower than the CPU (0.101 ms). The kernel alone takes only 0.143 ms, so roughly 93% of the elapsed time was spent on overhead rather than computation: the host-to-device copy, the device-to-host copy over the PCIe bus, device memory allocation and kernel launch latency. With only 100 elements there is almost nothing for the GPU's cores to parallelize, so this fixed cost dominates.

## SECTION 3: THREAD GEOMETRY & BOUNDARY MEMORY CORRUPTION (25 MIN)

**HARDWARE ANOMALY INSPECTION**

OBSERVATION REQUIREMENT:

Run the defective code snippet in Colab. In Python/Numba, write out of bounds on GPU memory often does not throw a Python IndexError! Why? Because GPU memory is raw VRAM. Writing out of bounds either silently corrupts adjacent GPU memory buffers or causes an asynchronous CUDA_ERROR_ILLEGAL_ADDRESS when you call cuda.synchronize().



Write down what exact error message or console behavior was displayed on your screen:

Observed behavior / Console output from defective kernel:

[ Write here:

_____

____

_____

_____ ]

### Task 3.2: Spatial Thread Coordinate Inspector (N = 37)

**Task 3.1 (observed behavior):**

No error was raised. The kernel finished normally and printed “Execution finished”, then “Elements processed: 1000. Last element: 999.0”. The 24 extra threads (IDs 1000–1023) wrote past the end of the array into adjacent raw VRAM without any Python IndexError or CUDA error, so the corruption was silent.

**Task 3.2:**

| Item | Value |
|---|---|
| Total blocks allocated | 5 |
| Total hardware threads launched | 40 |
| Active working threads | 37 |
| Idle guarded threads | 3 |

## SECTION 4: THE RACE CONDITION CRIME SCENE (PARALLEL REDUCTION) (25 MIN)

A race condition occurs when multiple threads attempt to read and write to the same memory address concurrently without synchronization. In this task, you will witness a massive data corruption event firsthand by attempting to sum an array in parallel.



**Task 4.1: The Naive Reduction Bug (Data Corruption)**

```
Trial 1: 2.0 | lost = 49998.0 | 0.175 ms
Trial 2: 2.0 | lost = 49998.0 | 0.165 ms
Trial 3: 2.0 | lost = 49998.0 | 0.118 ms
Trial 4: 2.0 | lost = 49998.0 | 0.119 ms
Trial 5: 2.0 | lost = 49998.0 | 0.106 ms
Atomic: 50000.0 0.474 ms
```

---

### Atomic results

- **Atomic Sum Output:** 50000.0. **Does it equal exactly 50,000.0?** Yes.

- **Naive Kernel Time:** about 0.106 to 0.175 ms. You can record the trial 5 value, **0.106 ms**, or the average of the five trials (about **0.137 ms**).
- **Atomic Kernel Time: 0.474 ms**.

The atomic kernel is about 3 to 4 times slower than the naive one (0.474 / 0.137 ≈ 3.5x).

Your atomic time came from a single run, so it may include some one-time overhead. If you want a steadier number, run the atomic cell two or three times and use the lowest value.

### Why atomic operations incur a penalty (ready to copy)

> Atomic operations are slower because all 50,000 threads target the same memory address, `total[0]`. The hardware must serialize the read-modify-write steps so only one thread updates that location at a time, which removes most of the parallelism and creates heavy memory contention. The naive kernel looks faster (about 0.14 ms vs 0.47 ms) only because it is wrong: threads overwrite each other's updates, so it returned 2.0 instead of 50,000.0. The atomic kernel pays extra time to give the correct answer.

### Remaining items

1. **Corner value:** run `print(result_canvas[0, 0])`. Expect 0.0.
2. **Section 6:** run the fixed checksum function with your real student ID and `CROSSOVER_N = 10000`. Make sure `d_canvas` still exists from Section 5 first.

---

## SECTION 5: 2D GRID IMAGE/MATRIX PROCESSING (20 MIN)

- Blocks in X: **64**, Blocks in Y: **64**, Total blocks: **4096**
- Center (512, 512): **1.000** (your output confirms it)
- Corner (0, 0): **0.000**. The corner's distance to the center equals `max_dist`, so intensity = 1 − 1 = 0. Confirm it with `print(result_canvas[0, 0])`.

## SECTION 6: FINAL AI-IMMUNE VERIFICATION CHECKSUM (10 MIN)



**FINAL SUBMISSION TOKEN & INTEGRITY CHECK**

PASTE YOUR UNIQUE 16-CHARACTER VERIFICATION TOKEN HERE:

[ TOKEN: `F81F12D38F09EF7E` ]

FINAL SUBMISSION CHECKLIST:

[ ] Notebook saved as CUDA_Lab01_<StudentID>.ipynb in Google Drive

[ ] All code cells executed sequentially with output clear and visible (no cleared outputs)

[ ] Exported as .ipynb (File → Download → Download .ipynb) and uploaded to course portal

[ ] This task sheet filled out completely and submitted to the instructor